

NextRupree — Master Build → Test → Deploy Document

Version 1.0 · July 2026 · Owner: DEN *Supersedes: FIOS_PRD.md (vision reference), FIOS_ML_Learning_Project.md (retired)*

1. Product

NextRupree — you tell it your money situation, it tells you the single best next use of your money (pay debt, build safety, buy gold, invest) and proves *why* with live data. An AI that referees the fight between what your family says, what your emotions say, and what the arithmetic says — and always shows its work.

Hero scenario (drives the landing page, the demo, and the golden set):

It's October. You have ₹60K spare. Family says buy gold for Diwali. Your credit card is at 42% APR. What should you actually do?

Demo in 90 seconds: 3-minute profile → 3-5 ranked NBCA cards → expand a card to see why-chain, confidence, invalidators, opportunity cost → ask “why is gold expensive right now?” and get a cited, live-data answer.

In scope (v1)

Profile intake · deterministic ranking engine · Gold Intelligence Engine (live data + knowledge graph + RAG) · LLM explanation layer with structured output · “ask why” Q&A · guardrails · eval harness in CI · deployed public demo.

Out of scope (documented, not built)

Forecasting models, agents, fine-tuning, other engines, accounts/auth beyond demo needs, payments, outcome-calibration ledger (named as future work in README), any specific-instrument recommendation, any execution of transactions.

2. Architecture & Stack

```
Next.js (Vercel)
├── REST/JSON
FastAPI (Render)
├── Rules Engine (pure Python, zero LLM)           ← owns ranking & all math
├── Explanation Layer (Claude/OpenAI API)          ← fills NBCA schema, never computes
├── Gold Engine
│   ├── Data feeds (FRED, yfinance, gold spot)
│   ├── Driver graph (NetworkX, ~30 nodes)
│   └── RAG (Chroma, ~50 curated docs)
└── Guardrail middleware
Supabase Postgres ← profiles (versioned), nbca_log (audit)
GitHub Actions    ← lint, unit tests, tiered evals on every PR
```

Locked choices: Python 3.12, FastAPI, Pydantic v2, Chroma (local persistent), NetworkX, Supabase, Next.js 14 + Tailwind, Claude Sonnet (demo) / Haiku (CI judge), GitHub Actions, Vercel + Render free tiers.

Non-negotiable principles: (1) all arithmetic in code — the LLM never computes; (2) every LLM output is schema-validated or rejected; (3) every claim in an explanation cites either a rules-

engine number or a retrieved document; (4) rankings are reproducible — same profile in, same ranking out.

3. Data Model (Supabase)

```
profiles (  
  id uuid PK, session_id text, version int,           -- append-only; never UPDATE  
  monthly_income numeric, monthly_expenses numeric,  
  cash_savings numeric, hi_debt_amount numeric, hi_debt_apr numeric,  
  dependents bool, term_insurance bool, risk_tolerance text,  
  goals jsonb, created_at timestampz  
)  
nbca_log (  
  id uuid PK, profile_id FK, rank int,  
  nbca jsonb,                                     -- full typed NBCA object  
  model text, prompt_version text, engine_data_asof date,  
  created_at timestampz  
)  
eval_runs (  
  id uuid PK, git_sha text, tier text, metrics jsonb, created_at timestampz  
)
```

Append-only profiles gives “what changed since last time” as a query; nbca_log is the audit trail (FIOS P0-7) for free.

4. NBCA Schema (Pydantic — the contract everything is built around)

```
class Evidence(BaseModel):  
    claim: str  
    source: Literal["rules_engine", "gold_engine", "document"]  
    ref: str                                     # calc id, driver id, or doc chunk id  
    value: str | None                            # the actual number shown to the user  
  
class NBCA(BaseModel):  
    action: str                                  # imperative, no instrument names  
    category: Literal["stabilize", "emergency_fund", "debt", "insurance", "invest_allocation"]  
    amount_range: tuple[int, int]                # rupees, computed by rules engine  
    rationale_chain: list[str]                   # ordered causal steps  
    evidence: list[Evidence]                     # >=1, every number traceable  
    confidence: Literal["high", "medium", "low"]  
    confidence_basis: str  
    invalidation_conditions: list[str]           # >=1  
    alternatives: list[str]                      # >=1  
    opportunity_cost: str
```

Validation failure ⇒ one retry with error feedback ⇒ then deterministic fallback card (rules-engine text template, flagged degraded=true). The product never 500s because a model rambled.

5. Rules Engine Specification

Distress / stabilization mode — triggers if ANY of (constants in config.py, each unit-tested):

Trigger	Threshold
Surplus	$\text{monthly_income} - \text{monthly_expenses} \leq 0$
Debt service ratio	$\text{monthly_debt_interest} > 40\% \text{ of income}$
Runway	$\text{cash_savings} < 1 \times \text{monthly_expenses}$
Debt overhang	$\text{hi_debt_amount} > 6 \times \text{monthly_surplus}$

Effect: all invest_allocation cards suppressed; ranking becomes stabilize → runway-to-1-month → debt restructuring.

Normal-mode allocation hierarchy (fixed priority with computed scores):

1. Emergency fund to 3 months (if runway < 3) — priority score $\propto (3 - \text{months_runway})$.
2. Insurance gap (dependents $\wedge$ no term cover) — cover heuristic $15 \times$ annual income.
3. High-interest debt prepay (APR $\geq 10\%$) — guaranteed-return framing: APR vs. expected asset-class return table.
4. Emergency fund 3→6 months.
5. Invest surplus — asset-class split by risk tolerance: low 35/50/15, med 55/30/15, high 70/20/10 (equity/fixed-income/gold). Gold share adjusted ± 5 pts by gold-engine regime signal (the one place the engine touches ranking, and it's bounded).

Every rule emits Evidence(source="rules_engine") objects with its computed numbers — this is what the LLM cites.

6. Gold Intelligence Engine

Live drivers (fetched daily, cached, asof-stamped): gold spot (USD & INR), USD index, US 10y real yield (FRED: DFII10), India CPI, INR/USD, Indian import duty (config constant, manually updated), domestic premium over landed price.

Driver graph (NetworkX, ~30 nodes): real_yields →(-) gold_usd, usd_index →(-) gold_usd, inr_depreciation →(+) gold_inr, import_duty →(+) local_premium, festival_season →(+) local_demand, etc. Each edge: direction, strength (qualitative), historical note, doc refs.

RAG corpus (~50 curated docs, static, ingested once): WGC reports, RBI/SGB scheme docs, import-duty history, 4-5 historical-regime writeups you author (2008, 2013 duty hikes, 2020, 2022 rate cycle — writing these teaches you the domain and makes retrieval demonstrably yours).

Retrieval: query → graph lookup (which drivers relate?) → Chroma vector search filtered to those drivers' doc tags → top-k chunks with ids. Graph-then-vector is the differentiator vs. generic RAG.

Freshness rule: any driver > 7 days stale = engine answers are flagged “based on data as of {date}” and confidence caps at medium. Stale > 30 days = engine facts excluded, fallback to structural knowledge only.

7. LLM Layer & Guardrails

- One prompt per job, versioned in repo (prompts/ with changelog): card explanation, why-Q&A, refusal.
- Tool-calling / JSON mode against the NBCA schema; temperature 0.2.
- **Guardrail middleware (pre- and post-model):**
 - Input: specific-instrument requests (“which stock/fund/coin”) → refusal template that reframes to asset-class reasoning.
 - Output: regex + NER scan for tickers/fund names/token names = block & regenerate; numbers in output diffed against evidence values ⇒ mismatch blocks the card.

- Distress mode verified server-side after generation (belt and braces).

8. Build Plan (8 weeks, part-time; each week ends demo-able)

Wk	Build	Exit criteria
1	Repo, CI skeleton (lint+pytest), Supabase schema, profile intake API, NBCA Pydantic schema	Profile POST→stored; schema round-trips; CI green
2	Rules engine complete: distress triggers, hierarchy, evidence emission	30+ unit tests incl. every distress boundary; identical-input⇒identical-ranking property test
3	Gold data feeds + driver graph + freshness logic	Daily fetch job runs; asof stamping; stale-data behavior tested
4	RAG: author regime docs, ingest, graph-then-vector retrieval	20 retrieval spot-checks pass; citations carry chunk ids
5	LLM explanation layer + guardrail middleware	Golden profile → 5 valid NBCA cards; instrument-request refusal works; fallback card path tested
6	Frontend: intake flow, NBCA cards w/ progressive disclosure, why-Q&A box, hero scenario on landing	Full user journey click-through on staging
7	Eval harness (all tiers, \$9) wired into CI; golden set authored	PR gate live: a deliberately-broken prompt fails CI
8	Deploy prod (\$10), polish, README w/ architecture diagram + eval results table, 90-sec demo script	Public URL; cold-start < 5s handled; README complete

Buffer: nothing new after week 8 — only fixes. Scope cuts if behind: why-Q&A box (week 6) is the first thing to drop; the cards alone still demo the product.

9. Test Plan

Tier 0 — deterministic (every PR, ~free, seconds): - Unit tests: rules engine math, every distress boundary (± 1 rupee cases), schema validation, guardrail regexes. - Golden-set ranking agreement: 50 expert-authored profiles → expected category ranking. **Zero LLM calls** — rankings are deterministic. Target: 100% (it's your own hierarchy; failures are bugs, not model drift). - Property tests: reproducibility, no `invest_allocation` card ever present when any distress trigger fires.

Tier 1 — cheap-model checks (every PR, ~cents, ~2 min): - Schema-fill success rate over 15 sampled profiles (target $\geq 95\%$ without fallback). - Guardrail suite: 25 adversarial prompts (stock picks, “guarantee me returns”, jailbreak phrasings) → 100% refusal, judged by Haiku/4o-mini. -

Grounding check: every Evidence.value in output must match a rules-engine or engine number exactly (string diff — deterministic, no judge needed).

Tier 2 — full eval (nightly + pre-release, demo model): - Explanation quality: LLM-judge rubric (causal chain coherent? invalidators concrete? opportunity cost quantified?) over full golden set; track score trend in eval_runs. - **Annotation-relative calibration:** confidence labels vs. expert-annotated confidence on golden set; report agreement matrix. *README states explicitly: this is agreement with annotation, not live outcome calibration — that requires the longitudinal calibration ledger, documented as future work.*

Manual (weekly, 15 min): 5 fresh profiles end-to-end in prod; read every word of one card critically.

CI wiring: Tier 0+1 block merge; Tier 2 nightly cron posts metrics to eval_runs and fails loudly on >5pt regression. Cost ceiling: ~\$0.10/PR, ~\$2/night — stated in README (cost-awareness is itself the signal).

10. Deploy Plan

Environments: local (Docker Compose: API + Chroma + Supabase CLI) → staging (Render preview + Vercel preview per PR) → prod.

Pipeline (GitHub Actions): 1. PR: lint → Tier 0 → Tier 1 → build Docker image → deploy previews. 2. Merge to main: auto-deploy Render (API) + Vercel (frontend); run smoke test (health check + 1 golden profile end-to-end); auto-rollback on smoke failure (Render previous-deploy API). 3. Nightly cron: data-feed refresh job + Tier 2 evals.

Config/secrets: all keys in Render/Vercel env vars + GitHub secrets; .env.example in repo; no secret ever in code or prompt files.

Prod hardening (portfolio-sized, deliberately minimal): rate limit 10 req/min/IP; per-day LLM spend cap with graceful “demo budget reached” message; Sentry free tier for errors; uptime ping (cron-job.org); /health endpoint reporting data freshness per driver.

Demo resilience: Render free tier sleeps — frontend shows a warm-up state on first request; a pre-computed cached demo profile renders instantly while live path wakes.

11. README / Portfolio Checklist (the artifact reviewers actually read)

- ☐ 2-line product statement + hero scenario at top
- ☐ Live demo link + 90-second GIF
- ☐ Architecture diagram (the §2 block, drawn)
- ☐ “Design decisions” section: LLM-never-does-math, graph-then-vector RAG, tiered evals-in-CI with cost table, honest calibration claim
- ☐ Eval results table (latest Tier 2 run, auto-generated)
- ☐ “What I’d build next” — calibration ledger, second engine via the FIOS engine interface, account aggregation
- ☐ Disclaimer: educational decision-support, not investment advice, no specific instruments — ever

One-line pitch: “A financial decision engine where the LLM is never trusted with math, every recommendation is falsifiable, and CI fails if answer quality regresses.”

12. Failure Modes & Mitigations

Failure	Mitigation
LLM output fails schema twice	Deterministic fallback card, degraded flag, logged
Data feed breaks	Freshness flags, confidence cap, structural-knowledge fallback (\$6)
Model names an instrument	Post-generation scanner blocks & regenerates; eval suite guards regression
Eval costs creep	Tiering + per-PR budget assertion in CI
Scope creep (the real killer)	Nothing new after week 8; second engine explicitly deferred; parking lot at end of README